

DemoEvolve: 利用演示克服 智能体框架进化中的稀疏反馈

Lirong Che^{1,2} Yuzhe Yang² Peiwen Lin²
Chuang Wang² Xueqian Wang² Jian Su²
¹Tsinghua University ²AgiBot

摘要

智能体框架进化通过修改围绕冻结语言模型智能体的可执行结构来改进它们。我们将这一范式研究为一种样本高效的快速适应形式：智能体无需更新模型权重，而是通过改变其外部框架来获得特定任务的能力，同时保持基础模型的通用能力不变。先前的工作表明，自生成的轨迹可以支持框架搜索，这表明智能体可能通过实践获得新的任务能力。然而，在长时程随机环境中，自我实践变得脆弱：奖励稀疏，结果方差大，失败难以归因于具体的框架机制。我们引入了 DemoEvolve，一种基于演示引导的框架进化方法。当仅靠奖励的搜索过于宽泛和嘈杂时，有能力的人类轨迹可作为编码提议者的专家参考经验，指导框架层面的诊断和编辑。在 Liar's Dice 上的实验表明，当回合较短且失败可归因时，自我轨迹进化可以奏效。相比之下，Balatro 暴露了一个更困难的长时程随机机制，其中自我轨迹进化被稀疏反馈和候选选择噪声误导，而仅靠教程式的文本知识并不能带来稳定的改进。在相同的有限预算下，DemoEvolve 产生了更有效且可审计的框架编辑，并取得了更好的性能。总体而言，演示使稀疏反馈下的框架进化更具可诊断性、可定位性和稳定性。

1 引言

大语言模型智能体不仅由模型权重塑造，还由围绕它们的外部框架塑造 [38, 31, 33]。最近关于框架进化的工作将这些结构视为围绕冻结模型的可编辑程序：编码提议者检查先前的代码、分数和执行轨迹，然后编写一个新的候选框架 [12, 42, 15, 17]。这暗示了一个令人兴奋的可能性：智能体的改进可以被视为通过对外部框架进行自动化程序搜索来实现样本高效的快速适应，从而在不重新训练模型或避免权重空间干扰其通用能力的情况下增加特定任务的能力。

现有在利用进化机制方面的成功主要集中在以文本和代码为核心的领域，在这些领域中，现代大型语言模型已经具备了丰富的先验知识 [13, 21, 15, 17]。本文探讨的是，同样的范式是否能够支持那些并非大型语言模型原生任务的世界，在这些世界中，改进不仅仅需要激发潜在的知识：利用机制必须帮助在一个陌生的交互环境中构建状态、动作和反馈。因此，本文采用游戏作为这一转变的受控研究领域。固定的随机种子和明确的动作接口使得候选方案的比较在受控评估协议下具有可重复性，而随机事件和延迟的结果则使它们具有有意义的状态性和交互性。图 1 将回合制游戏定位为介于模型原生工件优化与开放式实时世界之间的中间领域。

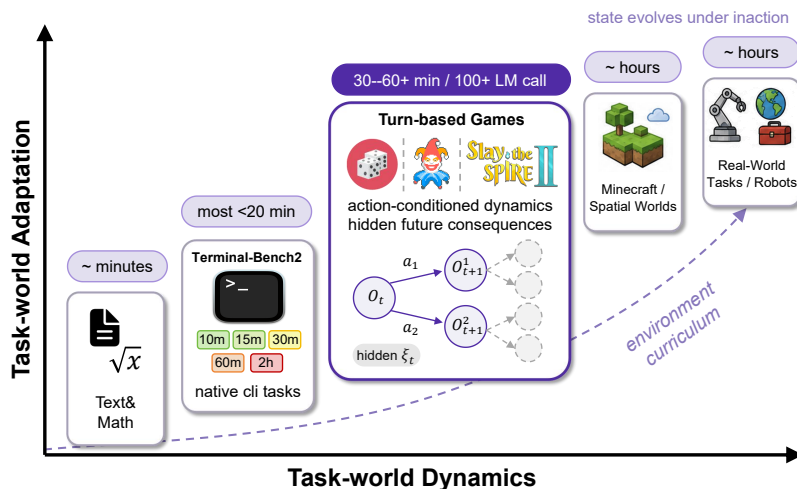


图 1：游戏作为任务世界适应的受控测试平台。本文根据任务世界动态和适应需求对领域进行定位。回合制游戏引入了以动作为条件、依赖于随机种子的未来以及稀疏的长期反馈，但通过固定随机种子和明确的动作接口保持了可重复性。因此，它们检验了利用机制进化是否能够超越模型原生先验，迈向任务特定的状态抽象和策略。

这一中间领域对利用机制进化背后的核心假设提出了挑战：先前的经验必须具有足够的诊断性，以指导下一次程序编辑。在短周期或模型原生任务中，这通常是可行的，因为失败相对局部，且迹较短。然而，在长期交互世界中，稀疏的结果可能被延迟且具有误导性。一次失败的运行可能包含数百个状态转换，早期的小决策可能重塑未来的状态分布，而随机性可能使一个无效的利用机制编辑偶然显得有益。因此，挑战不仅在于对利用机制程序的搜索，还在于信用分配：从嘈杂的轨迹级反馈中推断出应该改变哪个外部机制。

本文研究了大语言模型非原生博弈中的信用分配问题。我们使用文本竞技场吹牛骰子 [8] 作为短视界正控制，并使用小丑牌 [3, 2] 作为长视界随机环境。吹牛骰子表明，当失败是局部且可归因时，自我推演式框架演化能够发挥作用。相比之下，小丑牌揭示了一种机制，其中稀疏、高方差的反馈使得仅依赖奖励的搜索变得不可靠：表面上的得分提升可能并不对应于因果性的、面向模型的框架改进。这促使我们提出基于演示引导的框架演化，以使稀疏反馈更具可诊断性和可定位性。

总之，本文做出了以下贡献：

- 我们引入了演示演化，一种基于演示引导的框架演化方法。演示演化通过引入有能力的人类轨迹来增强自我推演档案，并将其用作框架级诊断和编辑的提议方参考经验。
- 我们揭示了长视界交互任务中自我推演框架演化的一种噪声选择失败模式：同种子推演方差和累积轨迹漂移可能使稀疏得分偏向于无效或非因果的框架编辑。
- 我们提供了小丑牌的证据，表明演示使稀疏反馈下的框架演化更加有效和可审计。在相同的有限预算下，演示演化产生了主动的面向模型的编辑，性能优于自我推演和文本引导的变体，并产生了与更好的编辑定位一致的行为和诊断证据。

2 相关工作

2.1 智能体框架优化与基于经验的自我改进

大语言模型智能体的性能不仅取决于模型权重，还取决于控制提示、工具使用、记忆和多步编排的外部执行结构。我们使用框架作为这些任务特定结构的总称，包括提示、工作流、工具，

记忆与执行逻辑。近期工作将此类结构视为可搜索或可编辑的对象，通过反馈驱动的重写、程序搜索、候选选择或外层循环进化来优化提示、工作流、工具、记忆或完整智能体框架 [12, 42, 30, 24, 18, 15, 17, 28, 41, 34, 35]。另一条相关研究路线通过将交互历史转化为可在后续任务中检索的可复用反思、记忆或技能来改进智能体 [31, 20, 44, 33, 39, 43, 40, 37, 19, 36]。这些工作通常假设过去的轨迹、反馈或执行日志包含可提取和复用的改进信号。而我们则探究在非模型原生、长时程交互任务中，自生成经验何时对框架进化仍然有用，其中失败可能稀疏、嘈杂且难以归因于具体的框架决策。

2.2 序贯决策中的经验分布塑造

在序贯决策中，学习受训练早期可用轨迹的强烈影响。模仿学习和从示范中学习利用专家行为提供参考轨迹，减少探索负担，并缓解专家状态与学习者诱导状态之间的分布不匹配 [29, 10, 9, 23, 27]。课程学习和自步学习进一步表明，控制训练经验的顺序、难度或目标可以改善优化稳定性和样本效率 [1, 14, 6]。针对稀疏奖励任务的相关探索方法也强调到达信息丰富或有前景的状态的重要性，而非从头开始均匀探索 [5]。我们的工作精神上相关，但不同之处在于大语言模型权重保持冻结，优化对象是外部框架；示范和对手条件轨迹作为框架级诊断和编辑的反馈，而非直接策略训练。

2.3 游戏与动态环境中的大语言模型智能体

游戏为静态问答和离线基准提供了互补的测试平台。近期基准将语言智能体置于文本游戏、视频游戏、棋牌游戏以及多智能体环境中，以评估动态状态跟踪、长程规划、隐藏信息推理、社交推理和对手建模 [8, 25, 16, 11, 4, 26, 7]。相关工作也开始使用随机类 Roguelike 牌组构建游戏，以揭示大语言模型智能体在长期资源管理、组合策略和风险控制方面的局限性 [3, 2]。与主要评估固定智能体或训练游戏策略的工作不同，我们将游戏用作测试环境，以考察在随机性、长程决策和动态交互下框架的自我进化。

3 方法

本节形式化了在大语言模型非原生任务世界中的演示引导框架进化。我们首先围绕冻结模型定义框架优化问题，然后区分模拟器层面的可复现性与智能体面临的不确定性。最后，我们描述外层循环编码提议器以及实验中比较的三种信息机制。

3.1 问题形式化

遵循元框架（Meta-Harness）[15]，我们将框架进化视为围绕冻结模型的可执行程序的端到端优化。设 W_T 表示任务世界， $x \sim \mathcal{X}_T$ 表示回合实例， π_ϕ 表示冻结模型， $h \in \mathcal{H}_T$ 表示面向任务的框架。任务世界暴露观测和任务特定的动作接口。在游戏中，这些动作可能包括出牌、弃牌、购买、出售或重掷等操作。框架决定观测如何表示、暴露哪些动作或工具、模型输出如何解析，以及动作如何在 W_T 中执行。 W_T, π_ϕ 和 h 共同诱导出交互轨迹上的展开分布。我们优化

$$h^* = \arg \max_{h \in \mathcal{H}_T} \mathbb{E}_{x \sim \mathcal{X}_T, \tau \sim p_{W_T, \pi_\phi, h}(\cdot | x)} [R_T(\tau, x)].$$

此处， $\tau = (o_0, a_0, o_1, a_1, \dots, o_T)$ 是任务世界交互轨迹， $R_T(\tau, x)$ 是稀疏任务奖励，通常仅在展开后观测到。模型参数 ϕ 始终保持固定；所有改进均来自改变可执行框架 h 。

受控的任务世界动态。我们的目标是研究在足够动态、需要状态抽象和长程动作推理，但又足够受控的任务世界中的框架进化

用于系统化搜索与评估。因此，本文将评估者对固定种子模拟器的视角与通过测试框架暴露给智能体的视角区分开来。设 $\tilde{s}_t$ 表示完整的任务世界状态，设 $o_t = \Omega(\tilde{s}_t)$ 为展示给智能体的观测，设 ρ 为回合种子。在固定种子下，完整状态转移可能是确定性的， $\tilde{s}_{t+1} = F_\rho(\tilde{s}_t, a_t)$ ，使得在同一种子上的重复评估可复现。

然而，评估者的可复现性并不意味着智能体知晓其动作的未来后果。智能体基于其观测-动作历史 $\eta_t = (o_0, a_0, \dots, o_t)$ 行动，而非基于完整的任务世界状态。因此，在智能体接口处，下一观测仍具有不确定性： $p_\rho(o_{t+1} | \eta_t, a_t) = \sum_{\tilde{s} \in \mathcal{S}} p_\rho(\tilde{s}_t = \tilde{s} | \eta_t) \mathbf{1}[o_{t+1} = \Omega(F_\rho(\tilde{s}, a_t))]$ 。

本文使用智能体侧不确定性来指代在观测与决策接口处产生的这种不确定性，而非所有模拟器变量固定后的内在非确定性。因此，回合制游戏提供了一种受控的动态机制：它们引入了动作条件化的未来后果与延迟反馈，同时保留固定种子和显式动作接口，以实现可复现的测试框架演化。

程序级诊断信用分配。由此产生的优化问题不同于标准的游戏学习。在强化学习中，稀疏结果通常通过预定义的学习规则 [32, 22] 与策略或价值更新相关联。在本文的设置中，被优化的对象是不可微的可执行测试框架：提示、状态跟踪、动作过滤、工具暴露和控制流。因此，终端奖励无法为特定的测试框架编辑提供直接的更新目标。提议者必须从轨迹、执行日志和稀疏奖励中推断出哪些测试框架机制导致了后续的成功或失败。

当回滚分数稀疏且嘈杂时，这种诊断问题十分脆弱。终端结果可能表明一次运行失败了，但无法说明失败是源于缺失的状态抽象、糟糕的动作过滤、薄弱的提示渲染、不恰当的工具暴露还是控制流错误。此外，一个候选方案可能因为轨迹方差或种子特定的选择噪声而显得更优，而非因为其编辑具有因果作用。因此，本文将分数提升与功能性测试框架改进区分开来：后者要求新的测试框架逻辑以可审计的方式改变面向模型的接口或执行行为。

3.2 不同信息机制下的测试框架演化

本文利用外层循环提议器 P 对 h 进行优化。该提议器是一个编码智能体，而非任务执行智能体本身。在每次迭代中，它检查先前的测试框架及其评估工件，诊断可能的失效模式，并编写一个或多个候选测试框架。

遵循元测试框架搜索循环 [15]，本文维护一个不断增长的文件系统档案 $\mathcal{D}_t$ 。每个经过评估的测试框架都会贡献一个评估目录，其中包含其源代码、开发集得分、执行轨迹以及原始执行迹。执行轨迹记录环境层面的交互 τ ，而原始执行迹则记录测试框架层面的细节，例如展示给模型的上下文、模型输出、解析后的动作、工具调用、错误以及测试框架侧的日志。提议器通过常规文件操作查询该档案，而非接收压缩后的摘要。

为便于符号表示，令 k_t 为外层循环迭代 t 中提出的候选数量。更新方式为

$$\{h^{(t+1,j)}\}_{j=1}^{k_t} \leftarrow P(I_t), \quad \mathcal{D}_{t+1} = \mathcal{D}_t \cup \{\text{EvalDir}(h^{(t+1,j)})\}_{j=1}^{k_t},$$

其中 I_t 表示提议器可用的信息， $\text{EvalDir}(h^{(t+1,j)})$ 是在开发实例上运行候选 $h^{(t+1,j)}$ 后写入的评估目录。最终测试实例被保留，在演化过程中对提议器不可见。

本文比较了三种信息机制：

$$I_t^{\text{meta}} = \mathcal{D}_t, \quad I_t^{\text{open}} = \mathcal{D}_t \cup K^{\text{text}}, \quad I_t^{\text{demo}} = \mathcal{D}_t \cup T^{\text{human}}.$$

图 2 总结了外层循环过程以及每种机制下可用的信息。

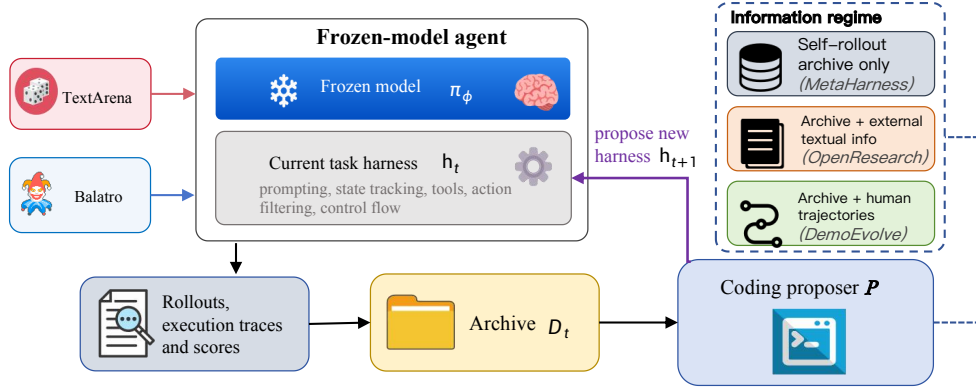


图 2: DemoEvolve 信息机制。编码提议器围绕一个冻结模型，利用包含先前执行轨迹、原始执行迹和得分的文件系统档案来演化任务测试框架。本文仅改变提议器可见的信息：自执行轨迹档案、档案加外部文本信息，或档案加人类轨迹。

元哈内斯 (Meta-Harness) 表示自滚动演化条件。在此机制下，提议者仅观察档案 $\mathcal{D}_t$ ：先前的哈内斯代码、得分、自生成的滚动轨迹以及原始执行迹。

开放研究 (OpenResearch) 表示文本引导的演化条件。除 $\mathcal{D}_t$ 外，提议者还接收外部文本任务信息 K^{text} ，例如规则、教程、策略指南或参考代码。该条件检验仅凭高层任务知识是否足以实现长时程哈内斯改进。

演示演化 (DemoEvolve) 表示我们的演示引导演化条件。除 $\mathcal{D}_t$ 外，提议者还接收熟练人类轨迹

$$T^{\text{human}} = \{(\tau_j^H, R_T(\tau_j^H, x_j))\}_{j=1}^m, \quad \tau_j^H = (o_0^H, a_0^H, o_1^H, a_1^H, \dots, o_{T_j}^H).$$

人类轨迹以与智能体滚动相同的任务世界观察-动作格式表示，但其动作来自熟练人类操作。我们将这些轨迹用作编码提议者的行为示例样本和诊断参考，而非监督式动作级标签或模型微调数据。由于它们同时暴露了熟练动作及这些动作发生的具体状态，有助于提议者将失败的智能体滚动与熟练行为进行对比，并定位缺失的状态抽象、不当的动作优先级、薄弱的资源管理或工具使用问题。

4 实验

我们利用游戏来检验线束演化能否将实践转化为围绕冻结模型的可执行任务启发式。这是一个自然的目标：许多游戏技能，对人类而言也是如此，都是作为经验法则习得的——何时跟注、加注、弃牌、保留或重掷——而非穷举规划。TextArena 说谎者的骰子 [8] 是乐观情形：尽管具有随机性和隐藏信息，有用的玩法仍可由紧凑的局部启发式捕捉，且自我推演演化能够从自身推演中发现此类脚手架。Balatro [3, 2] 是更困难的情形：有用的启发式存在，但其价值依赖于阶段和状态，因此稀疏的最终得分可能奖励错误的线束编辑。因此，我们利用 Balatro 来检验人类示范是否使该搜索更可诊断和可定位。

4.1 说谎者的骰子：自我推演发现紧凑启发式

设置。我们首先在 TextArena 说谎者的骰子 [8] 中评估线束演化，这是一个轻量级交互游戏，具有随机掷骰、隐藏信息、对手交互和顺序决策。该环境在更长时域的 Balatro 设置之前充当阳性对照：当推演反馈足够可归因时，提议者应能将观察到的失败转化为围绕冻结模型的可执行线束结构。

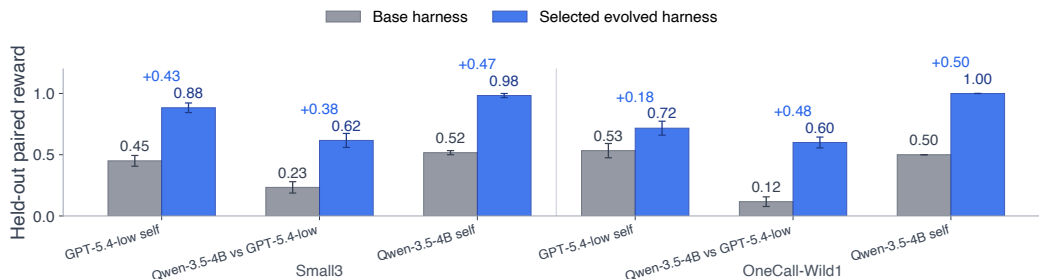


图 3: 仅使用开发集线束选择后的留出 TextArena 性能。每个柱状条使用 30 对逻辑留出种子。误差条显示标准误。柱状条标签显示平均留出奖励，每对上方数字为演化后减去基础奖励的差值。所有六个主要比较在留出种子上均有提升。

我们使用两种说谎者的骰子变体。Small3 是一种三骰顺序叫价游戏，而 OneCall-Wild1 是一种五骰单次叫牌变体，其中一点为万能牌。对于每个任务，我们评估三个目标/对手单元：GPT-5.4-low 自我博弈、Qwen-3.5-4B 对 GPT-5.4-low，以及 Qwen-3.5-4B 自我博弈。候选线束仅从开发/搜索推理中选择。留出评估在不相交种子上选择后进行，使用配对侧控制，每个比较 30 个逻辑留出种子。

结果。搜索循环在两项任务及所有模型配对中，均以少量提案找到了开发奖励更高的框架；完整开发曲线见附录 A。随后在留出种子上评估所选框架。如图 3 所示，任务平衡的宏观留出奖励从基础框架的 0.392 提升至所选进化框架的 0.800，绝对增益为 +0.408。该提升在各任务间保持一致：Small3 从 0.400 提升至 0.828，OneCall-Wild1 从 0.383 提升至 0.772。所有六项干净的运行级比较在留出种子上均有提升。

机制。对所选框架的检查表明，增益并非仅仅来自更长的提示。进化后的框架在冻结模型周围反复添加任务特定结构：观测解析、合法动作守卫、出价修复、概率估计、期望值跟注阈值、开局策略以及保守安全覆盖。

附录 B 提供了所选说谎者骰子框架的定性案例研究。案例表明，框架进化使模型-框架边界适应基础模型和任务变体：在一次 Qwen Small3 运行中，所选框架成为近符号化阈值策略，而更强模型和更难变体的运行则保留神经-符号分工，其中代码处理合法性、概率和风险，模型处理剩余的战略判断。

解读。这些结果表明，自滚动进化可以将实践转化为冻结模型周围可执行的局部启发式。说谎者骰子是这一模式的乐观案例：有用的玩法可以被紧凑的经验法则捕获，滚动轨迹为提议者提供了足够的信号，将脆弱的游戏特定计算外部化为确定性框架。Balatro 测试了这一模式的极限，其中类似启发式必须跨阶段和状态选择性应用，稀疏的最终分数成为框架搜索的更嘈杂指南。

4.2 Balatro: 稀疏反馈可能误导长时程框架搜索

设置。Balatro 是一款扑克牌组构建类 Roguelite 游戏 [3, 2]。我们将其用作长时程随机测试平台，因为成功游戏需要在种子相关的抽牌、商店、Boss 盲注和小丑协同作用下进行延迟资源管理。这些特性使得终端反馈稀疏、高方差，且难以归因于具体的框架机制。

我们采用 BalatroBench 中的固定种子设置 [2]。种子 A/B/C 在进化过程中用作开发种子，而种子 D/E 被保留并仅用于最终评估。候选框架仅从开发种子的运行结果中选取，选定的框架在所有五个种子上重新评估。

表 1: Balatro 在分布内 (ID) 和分布外 (OOD) 种子上的结果。每个单元格报告封顶的平均最终轮数, 括号内为完成率。

Method	ID seeds			OOD seeds		Averages		
	A	B	C	D	E	ID	OOD	Overall
BalatroLLM	12.0(0/3)	22.0(1/3)	17.0(2/3)	12.7(0/3)	21.0(2/3)	17.00(3/9)	16.83(2/6)	16.93(5/15)
Meta-Harness	16.7(0/3)	23.0(2/3)	18.3(2/3)	13.7(0/3)	19.0(2/3)	19.33(4/9)	16.33(2/6)	18.13(6/15)
OpenResearch	17.3(0/3)	15.0(1/3)	17.7(1/3)	14.0(0/3)	20.0(2/3)	16.67(2/9)	17.00(2/6)	16.80(4/15)
DemoEvolve	22.0(2/3)	24.0(3/3)	24.0(3/3)	16.0(1/3)	24.0(3/3)	23.33(8/9)	20.00(4/6)	22.00(12/15)

种子。完整的种子字符串、运行次数、选择规则、模型设置和成本细节见附录 C.2。

我们比较四种条件。**BalatroLLM** 是固定的 **BalatroLLM** 框架, 不进行外层循环进化 [3]。**Meta-Harness** 是自运行进化条件, 其中编码提议者仅看到先前的框架代码、分数、自生成的运行记录和执行迹 [15]。**OpenResearch** 添加外部文本任务信息, 如规则、教程、策略指南或参考代码。**DemoEvolve** 添加与智能体运行记录相同观察-动作格式的人类专家轨迹, 使提议者在提出框架编辑时能够检查具体的状态-动作证据。在所有进化条件中, 初始框架、搜索预算、任务模型、提议者、选择规则和最终评估协议均保持不变; 预期的差异仅在于提议者可获得的信息。

结果。表 1 报告了固定种子的 Balatro 性能。BalatroLLM 完成了 5/15 次运行。Meta-Harness 是自运行进化条件, 显示出明显的 ID 种子增益, 将 ID 平均最终轮数从 17.00 提高到 19.33, 完成率从 3/9 提高到 4/9。然而, 这种增益并未迁移: OOD 平均轮数从 16.83 变为 16.33, 完成次数仍为 2/6。

因此, 我们审计所选框架是否进行了主动的模型面向更改。对于每个选定的框架, 我们检查实现并重放固定种子运行, 并添加额外日志记录, 检查添加的钩子是否触发、派生状态变量是否被计算和注入, 以及模型调用输入或动作接口是否发生变化。选定的 Meta-Harness 候选包含一个预期的动态钩子, 但由于实现错误, 该钩子从未被触发。因此, 其明显的 ID 增益并非经过验证的功能性框架改进, 而更适合解释为在高方差运行评估下的噪声候选选择; 附录 C.3 提供了渲染级别的审计。

OpenResearch 虽然加入了外部文本任务信息但没有人类轨迹, 也未能提升整体性能, 仅完成了 4/15 次运行。相比之下, DemoEvolve 同时提升了分布内 (ID) 和分布外 (OOD) 种子: 它完成了 8/9 次 ID 运行和 4/6 次 OOD 运行, 总计 12/15 次。相反, 我们的审计表明, 所选的 DemoEvolve 改动是活跃的且面向模型的, 引入了基于人类轨迹证据的状态和阶段相关信息。这些结果表明, Balatro 的主要问题不仅是得分低, 还包括归因不可靠: 稀疏的自我运行演化可能会选择不活跃或非因果的编辑, 而人类轨迹为测试框架层面的信用分配提供了更具诊断性的证据。

4.3 行为分析: 人类演示转移策略

最终性能结果表明 DemoEvolve 表现更好, 但并未说明行为发生了哪些变化。因此, 我们考察人类轨迹是否转移了熟练玩法的策略级模式。我们重点关注经济管理, 因为 Balatro 的成功要求智能体在商店之间保留金钱, 同时仍将资源转化为足够的得分能力。

在 Balatro 中, 精确的动作匹配并不是一个可靠的目标: 重掷、购买卡包、弃牌或出牌的价值取决于当前的小丑牌组、手牌等级、首领盲注、牌组构成和商店供应。因此, 我们使用经济曲线作为一种粗略但可衡量的行为探针, 来考察长期策略。

为了避免幸存者偏差, 我们计算所有初始运行中的进店金钱, 对已经失败的运行赋值为零。对于方法 m 、种子面板 s 和商店轮次 r , 令 $N_{m,s}$ 为

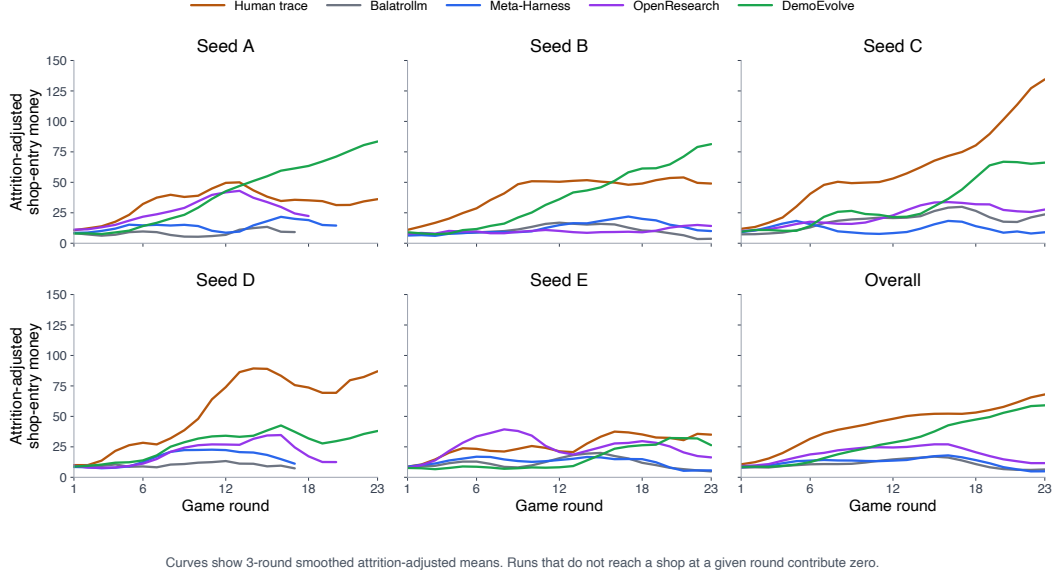


图 4: Balatro 中经损耗调整的进店经济。失败的运行在后续商店中贡献为零，因此后期数值同时反映生存能力和金钱管理。A/B/C 为分布内种子，D/E 为留出种子；D/E 上的人类轨迹仅为事后参考，在演化过程中不可见。右下角面板对所有种子取平均。细线显示原始调整均值，粗线显示三轮移动平均。

表 2: 经损耗调整的经济行为总结。进店指标由 $M_m(r)$ 计算；后期指标取第 19 – 23 轮的平均值。人类距离为与人类轨迹调整经济曲线的平均绝对差，数值越低表示行为越接近。

Method	Adj. entry mean	Late adj. entry	Late reach	Human dist.	Late human dist.
Human trajectories	42.99	61.87	1.00	0.00	0.00
BalatroLLM	10.89	6.61	0.39	32.10	55.25
Meta-Harness	12.16	6.97	0.45	30.83	54.89
OpenResearch	18.96	13.51	0.32	24.03	48.36
DemoEvolve	30.73	55.37	0.81	12.26	6.49

初始运行的数量。我们将调整后的进店金钱定义为

$$\widetilde{M}_{m,s}(r) = \frac{1}{N_{m,s}} \sum_{i=1}^{N_{m,s}} \widehat{M}_i(r), \quad \widehat{M}_i(r) = \begin{cases} M_i(r), & \text{if rollout } i \text{ reaches shop } r, \\ 0, & \text{otherwise.} \end{cases}$$

此处 $M_i(r)$ 是第 r 轮中 rollout i 的商店入场资金。因此，后期轮次的值同时反映了生存能力和经济管理能力。绘制的曲线应用了三轮移动平均以提高可读性；以下所有定量值均从未平滑的调整后曲线计算得出。

表 2 以数值形式展示了相同的模式。元线束（Meta-Harness）相对于 BalatroLLM 几乎未改变经济曲线：整体人类距离仅从 32.10 降至 30.83，且后期游戏距离仍然较高。开放研究（OpenResearch）整体上更接近人类曲线（24.03），但其后期触及范围仅为 0.32，因此该行为未能可靠地转化为生存能力。

演示进化（DemoEvolve）在性质上有所不同。其整体人类经济距离降至 12.26，后期游戏距离降至 6.49，且后期触及范围保持在 0.81。这支持了策略级迁移的解释：人类轨迹有助于进化后的线束支持类似人类的资源管理特征，而不仅仅是通过幸运的线束选择来提高最终得分。

4.4 定位分析：演示改进状态条件编辑定位

我们通过一项 5×3 诊断/设计研究来隔离修改定位，该研究与进化运行分开进行。所有实验组检查相同的固定 BalatroLLM rollout，仅在额外证据上有所不同：无、人类注释或人类轨迹。对于每个实验组，五个独立的提议者复现诊断故障并提出具体的编辑位置。

我们按上下文注入粒度对编辑进行分类。持久（Pers.）表示持久上下文；阶段（Phase）表示仅由环境提供的阶段字段门控的中间件；状态（State）表示在注入前由线束触发计算派生决策变量；工具（Tool）表示模型调用对当前状态的计算。我们报告计算（Comp）. 率 = （状态+工具） / 总数，即提供可执行决策支持的编辑所占比例。状态和工具在触发方式上有所不同：状态由线束注入，而工具将调用委托给模型。

表 3 将缺陷定位与修正信号区分开来。仅靠推演的诊断/设计研究看到的缺陷注入或解释条件具有较高的完成率（46.2%），但证据仅为负向。它显示了增强的推理能力，但并非在 Balatro 中如 rollout 所示。额外证据（extra.ev）表示附加证据，计算率（Comp. rate）统计状态和工具编辑。人类轨迹提供了正向的状态-动作反事实，恢复了可比的完成率（45.5%），同时将计算出的干预措施锚定在应注入或调用的观测条件上。

	额外证据	持久	阶段	状态	工具	计算率
None	7/26	7/26	10/26	2/26		46.2%
Notes	9/23	9/23	1/23	4/23		21.7%
Human trajectories	8/22	4/22	6/22	4/22		45.5%

经济管理说明了这一区别。仅靠推演的诊断看到的是事后症状，如超支、损失利息奖励或后期缓冲薄弱，因此其编辑变成了购买侧防护措施。人类笔记并非缺失原则：一个仅用笔记的复现提出了手牌选择的微提示，即未使用的手牌会成为回合末收入。缺失的信号是状态边界。人类轨迹显示，在一局早期、尚未出牌、剩余多次弃牌且无强得分手牌时，熟练玩家通常会先弃牌，然后以更少的手数清空该回合。这将原则转化为状态计算干预，例如由 `hands_played=0, 次弃牌  $\geq 2$` 和早期底注门控的 `round1_dig_hint`。因此，推演定位价值损失，笔记识别原则，而人类轨迹指定了何时应触发该干预机制。

5 结论

我们研究了在长时程、随机交互任务中冻结模型智能体的机制进化，其中稀疏的终端反馈和高推演方差使自我改进难以诊断。TextArena 表明，当失败是短时程且可归因时，自我推演进化可以奏效，而 Balatro 则暴露了一个更困难的机制，其中稀疏分数可能选择无效或非因果的编辑。DemoEvolve 表明，胜任的演示可以使这一搜索问题变得更容易：它们提供了正向的状态-动作证据，缩小了有效的机制搜索空间，并有助于将稀疏失败转化为可操作的、面向模型的机制。这些结果表明，演示可以使稀疏反馈下的机制进化更具样本效率、可诊断性和稳定性。

6 局限性

我们的主要局限性在于任务覆盖范围。演示进化主要在《小丑牌》中进行评估，而文本竞技场则作为在更可归因反馈下自滚动演化的轻量级阳性对照。尽管《小丑牌》是一个有用的长时程随机测试平台，但它仍然只是一个游戏环境。未来的工作应测试基于演示引导的测试框架演化是否能迁移到更广泛的交互式环境中，包括其他游戏、工具使用智能体、终端任务、网页智能体和具身环境。

参考文献

- [1] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th Annual International Conference on Machine Learning*, pages 41–48. Association for Computing Machinery, 2009.
- [2] Coder. Balatrobench. <https://balatrobench.com/>, 2026. Accessed May 2, 2026.
- [3] Coder. BalatroLLM: Play Balatro with LLMs. <https://github.com/coder/balatrollm>, 2026. Accessed May 2, 2026.
- [4] Jinhao Duan, Renming Zhang, James Diffenderfer, Bhavya Kailkhura, Lichao Sun, Elias Stengel-Eskin, Mohit Bansal, Tianlong Chen, and Kaidi Xu. Gtbench: Uncovering the strategic reasoning limitations of llms via game-theoretic evaluations, 2024. URL <https://arxiv.org/abs/2402.12348>.
- [5] Adrien Ecoffet, Joost Huizinga, Joel Lehman, Kenneth O. Stanley, and Jeff Clune. First return, then explore. *Nature*, 590(7847):580–586, 2021.
- [6] Carlos Florensa, David Held, Markus Wulfmeier, Michael Zhang, and Pieter Abbeel. Reverse curriculum generation for reinforcement learning. In *Conference on Robot Learning*, pages 482–495, 2017.
- [7] Lingyue Fu, Xin Ding, Linyue Pan, Yaoming Zhu, Shao Zhang, Lin Qiu, Weiwen Liu, Weinan Zhang, Xuezhi Cao, Xunliang Cai, Jiaxin Ding, and Yong Yu. Catarena: Evaluating evolutionary capabilities of code agents via iterative tournaments, 2025. URL <https://arxiv.org/abs/2510.26852>.
- [8] Leon Guertler, Bobby Cheng, Simon Yu, Bo Liu, Leshem Choshen, and Cheston Tan. Textarena, 2025. URL <https://arxiv.org/abs/2504.11442>.
- [9] Todd Hester, Mel Vecerik, Olivier Pietquin, Marc Lanctot, Tom Schaul, Bilal Piot, Jean-Baptiste Lespiau, Laurent Sartran, and Guillaume Beaudoin. Deep q-learning from demonstrations. *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1), 2018.
- [10] Jonathan Ho and Stefano Ermon. Generative adversarial imitation learning. In *Advances in Neural Information Processing Systems*, 2016.
- [11] Lanxiang Hu, Qiyu Li, Anze Xie, Nan Jiang, Ion Stoica, Haojian Jin, and Hao Zhang. Gamearena: Evaluating llm reasoning through live computer games, 2024. URL <https://arxiv.org/abs/2412.06394>.
- [12] Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. In *The Thirteenth International Conference on Learning Representations*, October 2024. URL <https://openreview.net/forum?id=t9U3LW7JVX>.
- [13] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, October 2023. URL <https://openreview.net/forum?id=VTF8yNQM66>.
- [14] M. Pawan Kumar, Benjamin Packer, and Daphne Koller. Self-paced learning for latent variable models. *Advances in Neural Information Processing Systems*, 23, 2010.
- [15] Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-harness: End-to-end optimization of model harnesses, March 2026. URL <https://arxiv.org/abs/2603.28052>.
- [16] Wenhao Li, Wenwu Li, Chuyun Shen, Junjie Sheng, Zixiao Huang, Di Wu, Yun Hua, Wei Yin, Xiangfeng Wang, Hongyuan Zha, and Bo Jin. Textatari: 100k frames game playing with language agents, 2025. URL <https://arxiv.org/abs/2506.04098>.
- [17] Jiahang Lin, Shichun Liu, Chengjun Pan, Lizhi Lin, Shiha Dou, Xuanjing Huang, Hang Yan, Zhenhua Han, and Tao Gui. Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses, April 2026. URL <https://arxiv.org/abs/2604.25850>.

- [18] Xinghua Lou, Miguel Lázaro-Gredilla, Antoine Dedieu, Carter Wendelken, Wolfgang Lehrach, and Kevin P. Murphy. Autoharness: improving llm agents by automatically synthesizing a code harness, 2026. URL <https://arxiv.org/abs/2603.03329>.
- [19] Ziyu Ma, Shidong Yang, Yuxiang Ji, Xucong Wang, Yong Wang, Yiming Hu, Tongwen Huang, and Xiangxiang Chu. Skillclaw: Let skills evolve collectively with agentic evolver, April 2026. URL <http://arxiv.org/abs/2604.08377>.
- [20] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In *Thirty-Seventh Conference on Neural Information Processing Systems*, November 2023. URL <https://openreview.net/forum?id=S37h0erQLB>.
- [21] Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Marianna Nezhurina, Jenia Jitsev, Di Lu, Orfeas Menis Mastromichalakis, Zhiwei Xu, Zizhao Chen, Yue Liu, Robert Zhang, Leon Liangyu Chen, Anurag Kashyap, Jan-Lucas Uslu, Jeffrey Li, Jianbo Wu, Minghao Yan, Song Bian, Vedang Sharma, Ke Sun, Steven Dillmann, Akshay Anand, Andrew Lanpouthakoun, Bardia Koopah, Changran Hu, Etash Guha, Gabriel H. S. Dreiman, Jiacheng Zhu, Karl Krauth, Li Zhong, Niklas Muennighoff, Robert Amanfu, Shangyin Tan, Shreyas Pimpalgaonkar, Tushar Aggarwal, Xiangning Lin, Xin Lan, Xuandong Zhao, Yiqing Liang, Yuanli Wang, Zilong Wang, Changzhi Zhou, David Heineman, Hange Liu, Harsh Trivedi, John Yang, Junhong Lin, Manish Shetty, Michael Yang, Nabil Omi, Negin Raoof, Shanda Li, Terry Yue Zhuo, Wuwei Lin, Yiwei Dai, Yuxin Wang, Wenhao Chai, Shang Zhou, Dariush Wahdany, Ziyu She, Jiaming Hu, Zhikang Dong, Yuxuan Zhu, Sasha Cui, Ahson Saiyed, Arinbjörn Kolbeinsson, Jesse Hu, Christopher Michael Rytting, Ryan Marten, Yixin Wang, Alex Dimakis, Andy Konwinski, and Ludwig Schmidt. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces, January 2026. URL <http://arxiv.org/abs/2601.11868>.
- [22] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015. doi: 10.1038/nature14236.
- [23] Ashvin Nair, Bob McGrew, Marcin Andrychowicz, Wojciech Zaremba, and Pieter Abbeel. Overcoming exploration in reinforcement learning with demonstrations. In *2018 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6292–6299, 2018.
- [24] Alexander Novikov, Ngán Vű, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. Alphaevolve: A coding agent for scientific and algorithmic discovery, June 2025. URL <http://arxiv.org/abs/2506.13131>.
- [25] Davide Paglieri, Bartłomiej Cupiał, Samuel Coward, Ulyana Piterbarg, Maciej Wolczyk, Akbir Khan, Eduardo Pignatelli, Lukasz Kucinski, Lerrel Pinto, Rob Fergus, Jakob Nicolaus Foerster, Jack Parker-Holder, and Tim Rocktaschel. Balrog: Benchmarking agentic llm and vlm reasoning on games, 2024. URL <https://arxiv.org/abs/2411.13543>.
- [26] Dongmin Park, Minkyu Kim, Beongjun Choi, Junhyuck Kim, Keon Lee, Jonghyun Lee, Inkyu Park, Byeong-Uk Lee, Jaeyoung Hwang, Jaewoo Ahn, Ameya S. Mahabaleshwar, Bilal Kartal, Pritam Biswas, Yoshi Suhara, Kangwook Lee, and Jaewoong Cho. Orak: A foundational benchmark for training and evaluating llm agents on diverse video games, 2025. URL <https://arxiv.org/abs/2506.03610>.
- [27] Aravind Rajeswaran, Vikash Kumar, Abhishek Gupta, Giulia Vezzani, John Schulman, Emanuel Todorov, and Sergey Levine. Learning complex dexterous manipulation with deep reinforcement learning and demonstrations. In *Robotics: Science and Systems*, 2018.

- [28] Maxime Robeyns, Martin Szummer, and Laurence Aitchison. A self-improving coding agent, 2025. URL <https://arxiv.org/abs/2504.15228>.
- [29] Stéphane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, pages 627–635, 2011.
- [30] Asankhaya Sharma. Openevolve: an open-source evolutionary coding agent. <https://github.com/algorithmicsuperintelligence/openevolve>, 2025. URL <https://github.com/algorithmicsuperintelligence/openevolve>. GitHub repository.
- [31] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik R. Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Thirty-Seventh Conference on Neural Information Processing Systems*, November 2023. URL <https://openreview.net/forum?id=vAElhFcKW6>.
- [32] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2 edition, 2018.
- [33] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models, October 2023. URL <http://arxiv.org/abs/2305.16291>.
- [34] Wenyi Wang, Piotr Piekos, Nanbo Li, Firas Laakom, Yimeng Chen, Mateusz Ostaszewski, Mingchen Zhuge, and Jurgen Schmidhuber. Huxley-godel machine: Human-level coding agent development by an approximation of the optimal self-improving machine, 2025. URL <https://arxiv.org/abs/2510.21614>.
- [35] Chunqiu Steven Xia, Zhe Wang, Yan Yang, Yuxiang Wei, and Lingming Zhang. Live-swe-agent: Can software engineering agents self-evolve on the fly?, 2025. URL <https://arxiv.org/abs/2511.13646>.
- [36] Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, Zeyu Zheng, Cihang Xie, and Huaxiu Yao. Skillrl: Evolving agents via recursive skill-augmented reinforcement learning, February 2026. URL <http://arxiv.org/abs/2602.08234>.
- [37] Yiming Xiong, Shengran Hu, and Jeff Clune. Learning to continually learn via meta-learning agentic memory designs. In *OpenReview*, 2026. URL <https://api.semanticscholar.org/CorpusID:285454009>.
- [38] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*, 2022. URL https://openreview.net/forum?id=WE_vluYUL-X.
- [39] Haoran Ye, Xuning He, Vincent Arak, Haonan Dong, and Guojie Song. Meta context engineering via agentic skill evolution. *arXiv preprint arXiv:2601.21557*, 2026.
- [40] Guibin Zhang, Haotian Ren, Chong Zhan, Zhenhong Zhou, Junhao Wang, He Zhu, Wangchunshu Zhou, and Shuicheng Yan. Memevolve: Meta-evolution of agent memory systems. *arXiv preprint arXiv:2512.18746*, 2025.
- [41] Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin godel machine: Open-ended evolution of self-improving agents, 2026. URL <https://arxiv.org/abs/2505.22954>.
- [42] Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xiong-Hui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. Aflow: Automating agentic workflow generation. In *The Thirteenth International Conference on Learning Representations*, October 2024. URL <https://openreview.net/forum?id=z5uVAKwmjf>.

- [43] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. Agentic context engineering: Evolving contexts for self-improving language models. In *The Fourteenth International Conference on Learning Representations*, October 2025. URL <https://openreview.net/forum?id=eC4ygDs02R>.
- [44] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: Llm agents are experiential learners, December 2024. URL <http://arxiv.org/abs/2308.10144>.

附录 A 文本竞技场说谎者骰子细节

本附录为第 4.1 节报告的文本竞技场说谎者骰子实验提供额外细节。这些实验作为轻量级阳性对照：在进入更长时程的《小丑牌》环境之前，测试当反馈相对短时程且可归因时，自滚动测试框架演化是否有效。

游戏变体。我们报告两种双人说谎者骰子变体。小 3 使用说谎者骰子-v0-小环境：每位玩家起始有三颗骰子，1 点不是万能牌，玩家交替叫出数量/点数组合或质疑上一个叫牌，输掉挑战则移除一颗骰子。一叫-万能 1 使用说谎者骰子-v0-一叫环境：每位玩家起始有五颗骰子，1 点在叫出 1 点之前是万能牌，首次 [质疑] 立即结束比赛，不损失骰子也不重掷。这些变体具有交互性和随机性，包含隐藏骰子、对手压力、合法行动约束和顺序叫牌决策。然而，与《小丑牌》相比，每个回合足够短，终局结果通常可以追溯到少数叫牌或质疑决策。因此，它们为测试自生成的滚动轨迹是否能支持测试框架编辑提供了更清晰的环境。

作为轻量级阳性对照的角色。我们使用文本竞技场说谎者骰子来检验当失败可归因且搜索信号相对密集时，外层循环机制能否改进冻结模型智能体。该实验的目的不是证明自滚动反馈足以应对稀疏、长时程游戏。相反，它确立了一个阳性案例：当任务时程较短且滚动失败易于定位时，自滚动测试框架演化可以将观察到的失败转化为有用的可执行测试框架结构。

搜索与选择。对于每个目标/对手/任务单元，外层循环提议器仅修改冻结目标模型周围的外部测试框架。候选选择仅使用开发/搜索运行数据；保留的种子在候选提议或选择期间不可见。搜索预算刻意较小：候选评估通常使用 $N = 5$ 个逻辑开发种子，而后续部分 Small3 候选和基线使用 $N = 10$ 个。所选测试框架仅根据开发奖励进行选择。一次 OneCall-Wild1 GPT 自我博弈运行采用了预先声明的仅开发平局决胜规则：两个候选均达到开发奖励 0.700，干净的平局决胜选择了后一个候选。该平局决胜未使用保留性能。

保留配对评估。最终评估对每个基线/进化比较使用 30 个匹配的逻辑保留种子。Small3 使用保留逻辑种子 120000 – 120029，OneCall-Wild1 使用保留逻辑种子 130000 – 130029。每个单元内基线测试框架和所选进化测试框架使用相同的种子集。

我们使用配对侧控制进行保留评估。每个逻辑种子评估为两局原始游戏，使用相同的环境种子并交换玩家位置。报告的配对奖励取两局平均值，因此胜利获得奖励 1，平局获得奖励 0.5，失败获得奖励 0。因此 $N = 30$ 表示每个基线或进化行有 30 场配对逻辑比赛，即 60 局原始游戏。这控制了先手和座位顺序效应，同时保持基线比较和进化比较中的随机骰子结果固定。

度量与干净运行标准。主要度量是 30 个逻辑种子上保留配对奖励的平均值，标准误基于配对奖励计算。论文层面的聚合是任务平衡的：我们分别对 Small3 和 OneCall-Wild1 内的运行级均值取平均，然后对两个任务均值取平均。在干净平局决胜包中，基线测试框架得到 0.392，所选进化测试框架得到 0.800，绝对增益为 +0.408。

仅当所有 30 个基线和 30 个进化配对摘要均存在、所有逻辑种子的配对侧评估成功、且目标侧或对手侧均无 LLM 回退、错误或超时事件时，该行才被纳入干净主结果。无效动作和解析回退记录为行为审计度量，而非基础设施故障。

工件再生。所报告的文本竞技场 (TextArena) 数值由归档的干净决胜工件生成，这些工件包含原始留出迹、留出摘要、宏摘要、审计文件、

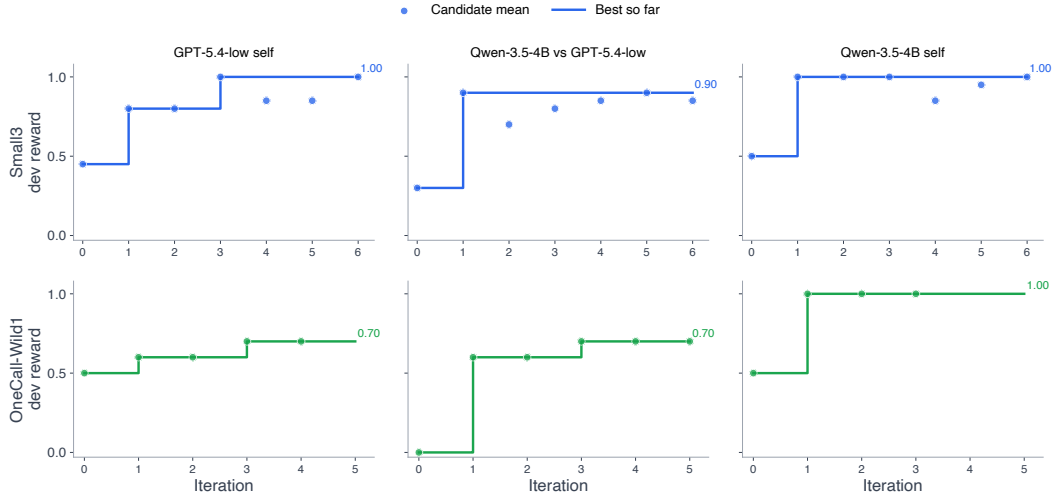


图 5：文本竞技场说谎者骰子的开发/搜索进展。迭代 0 为基础测试框架。点表示候选开发奖励，线表示迄今观察到的最佳开发奖励。候选选择仅使用开发/搜索展开；搜索过程中不使用留出种子。

源快照以及每个候选的搜索进展记录。执行并行仅控制并发，不会在同一随机种子上重复评估。与任何使用托管或本地服务的大语言模型（LLM）的评估一样，精确的逐位复现可能取决于模型服务实现、端点可用性和重试行为。因此，所报告的数值与归档的原始迹和摘要相关联，而种子协议则指定了用于复现的随机博弈实例。

B 演化说谎者骰子测试框架的定性分析

本节定性检查第 4.1 节文本竞技场实验中选定的说谎者骰子测试框架。目的不是展示完整源代码，而是理解自我展开测试框架演化发现了哪些可执行启发式方法。在所选的测试框架中，我们观察到一个反复出现的模式：外层循环将脆弱的决策计算从自由形式的语言模型推理中移出，并放入可执行脚手架中。根据基础模型和博弈变体的不同，这种脚手架的范围从近符号控制到神经网络分工。

案例 A：较弱模型的近符号阈值控制。在小 3 千问（Small3 Qwen）自我博弈设置中，选定的测试框架在决策时很大程度上绕过了语言模型。它解析观察结果，估计当前叫价是否为真，当叫价看起来足够不可能时进行质疑，否则以可接受的概率加注到最低合法叫价。简化示意如下：

```
状态 = parse_observation(observation)
```

```
如果 no_current_bid: 返回
opening_bid_from_own_dice(state)
```

```
p_true = P(current_bid_is_true | own_dice, unknown_opponent_dice)
```

```
如果 p_true < 0.30: 返回
“[呼叫]”
```

```
加注 = smallest_legal_raise_with_probability_at_least(0.50) 返回加注
```

该案例之所以有用，恰恰是因为它接近一个极限情况。框架（Harness）的演化并不受限于为语言模型保留固定角色。当基础模型在局部博弈机制上较弱或不稳定时，

搜索可以通过将更多决策过程转移到确定性代码中，来改进整体冻结模型智能体。在这种轻量级设置下，由此产生的“技能”更像是一种紧凑的经验法则策略，而非改进的模型推理：被优化的对象是整个模型-框架系统。

案例 B：对手感知的期望值评分。在 Small3 GPT-5.4 自我博弈设置中，选定的框架保持了更混合的结构。框架解析累积的观察历史，从对手当前轮的出价构建简单的对手面先验，计算当前声明和候选加注的二项式尾部概率，并按期望值对合法动作进行排序。简化示意如下：

```
opp_prior = compute_opp_face_prior(opponent_bids_this_round)

p_current = P(current_bid_true | own_dice, opp_dice_count, opp_prior)
ev_call = 2 * (1 - p_current) - 1

对于加注在 legal_raises(current_bid): p_raise = P(raise_true | own_dice, opp_dice_count,
opp_prior) ev_raise = 2 * p_raise - 1
```

返回 best_action_by_ev([调用] + legal_raises)

该框架不仅决定向模型展示何种文本，还计算模型在多轮交互中难以稳定维持的决策相关变量：合法加注、声明概率、对手条件先验以及跟注与加注之间的权衡。语言模型仍可用于残余的战略判断或动作表述，但脆弱的算术和合法性约束在模型外部得到稳定。

案例 C：OneCall-Wild1 中的贝叶斯堆叠防护。OneCall-Wild1 是更难的吹牛骰子变体，因为一点为万能牌，且一次叫牌即结束比赛。在选定的 GPT-5.4 框架中，演化逻辑为深度同面升级添加了状态条件防护。该框架估计当前叫牌的真实概率，在基于对手上次叫牌的贝叶斯堆叠模型下估计最佳后续加注，并在跟注的期望值高出一定幅度时覆盖继续加注的行为：

```
p_truth = P(current_bid_true | own_dice, wild_rule)
best_raise = argmax_raise P(raise_true | opponent_last_bid, wild_rule)

ev_call = 1.0 - p_truth
ev_raise = P(best_raise_true | bayesian_pile_on_model)
```

如果 own_contribution >= 2 且 ev_call > ev_raise + 幅度：动作 = "[跟注]"
否则：动作 = best_raise

该案例展示了一种更明确的神经符号模式。框架处理规则解释、概率估计、合法动作验证和高风险覆盖，同时仍允许在模糊状态下进行面向模型的分析或建议。该干预是状态条件化的而非种子条件化的：它针对叫牌轨迹中的结构模式触发，而非针对特定的开发种子。

要点。说谎者骰子案例研究表明，自我展开的框架进化能够发现紧凑且可执行的游戏启发式规则。在较简单或较弱模型的设置下，所选框架可能变得近乎符号化，用确定性阈值和守卫条件取代模型不稳定的局部决策。在更困难的变体或使用更强模型时，框架倾向于形成混合系统：符号代码处理显式规则、概率、合法动作和安全约束，而冻结的语言模型处理剩余的语义和战略判断。这支持了框架进化优化整个模型-框架系统行为的观点，而不仅仅是提示或模型独立生成质量。

C 小丑牌背景与术语

C.1 游戏规则与术语

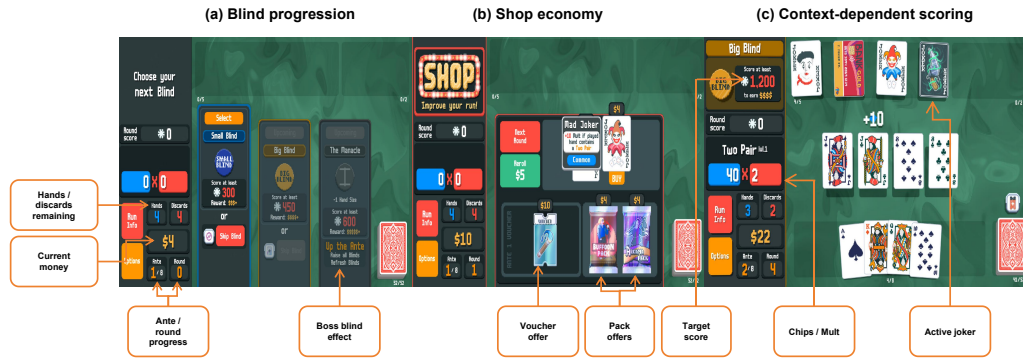


图 6：带注释的小丑牌截图，说明我们分析中使用的机制。(a) 盲注推进定义了生存目标：智能体必须在有限的手牌和弃牌次数内清除不断增加的目标分数，而首领盲注则附加特殊约束。(b) 商店阶段将金钱转化为长期资源，因为智能体必须在购买、重掷和为未来商店存钱之间做出选择。(c) 得分依赖于构建：同一手打出的牌可能因筹码、倍率、剩余资源和激活的小丑牌而有不同价值。

本节为未玩过该游戏的读者提供小丑牌的简要介绍。我们重点介绍解释我们的基准、度量和行为分析所需的机制。

运行结构。一局小丑牌由一系列底注组成，每个底注包含若干盲注。要清除一个盲注，玩家必须在耗尽可用手牌和弃牌次数之前达到目标分数。未能达到目标分数则运行结束。因此，我们使用一次展开的最终回合作为生存深度的粗略度量。

得分与出牌。在每个盲注阶段，玩家进行扑克式牌型。所打牌型的得分取决于牌型类别、牌面数值、牌型等级、倍率以及生效的修正效果。因此，最佳行动往往并非孤立的最强常规扑克牌型；它取决于当前的构筑、可用的成长效果以及剩余资源。

商店与经济。盲注阶段结束后，玩家进入商店阶段，可以花费金钱购买小丑牌、优惠券、补充包、消耗品或进行重掷。因此，金钱是一种战略资源，而非单纯的局部奖励信号。激进消费可能提升即时生存能力，但保留金钱可以促成更强的后续商店和更灵活的成长。这促使我们将进入商店时的金钱作为长期行为探针进行分析。

小丑牌与协同效应。小丑牌是持续生效的修正牌，通常决定了一局游戏的策略。其价值高度组合化：一张小丑牌可能单独较弱，仅在特定牌组编辑或消耗品配合下才强大，或者前期有用但后期价值降低。这种情境依赖性使得在不考虑当前小丑牌组合、牌组构成和游戏阶段的情况下，难以解读行动价值。

随机性与种子。小丑牌（Balatro）包含大量随机性，包括商店库存、补充包内容、首领盲注效果以及抽牌。固定种子可使一局游戏可复现，但在少量种子上反复优化仍可能利用特定种子的机会，而非学习普遍有效的策略。因此，我们区分用于框架演化的开发种子与仅用于最终评估的留出评估种子。

正文中使用的术语。主论文使用了若干《小丑牌》专属术语：

- **盲选 (Blind)：**一个必须清除才能继续运行的评分挑战。

- **底注 (Ante) :** 包含多个盲注 (Blind) 的运行阶段; 后续底注通常要求更高的分数。
- **商店入场费:** 盲注后进入商店时可用的金额。
- **小丑集合:** 当前塑造本轮得分、经济与成长行为的小丑牌集合。
- **首领盲注:** 一种带有额外约束或效果的盲注, 可以扰乱原本强大的战术。
- **重掷:** 花费金钱来刷新商店的商品。

C.2 实验方案

固定种子与数据划分。 本文采用 BalatroBench 中的固定种子设置 [2]。每个选定的测试框架在五个确定性的 Balatro 种子上进行评估, 每个种子进行三次独立运行。为便于阅读, 本文将具体的种子字符串 AAAAAAA - EEEEEEE 缩写为 A - E。种子 A/B/C 在进化过程中用作开发种子, 而种子 D/E 被保留并仅用于最终评估。在进化过程中, 提议者不会观察到来自 D/E 的运行结果、分数或迹。

搜索与选择。 对于每种进化条件, 本文从相同的 BalatroLLM 测试框架 [3] 开始, 并运行一个 10 次迭代的外层循环。在每次迭代中, 编码提议者通常产生两个候选测试框架。每个候选框架在搜索过程中在种子 A/B/C 上进行评估, 每个种子运行一次。遵循元测试框架协议 [15], 提议者可以检查先前测试框架的存档、分数、自生成的运行结果和执行迹, 并可以选择基于哪个先前版本进行构建。在搜索预算耗尽后, 本文选择在 A/B/C 上平均最终轮数最高的候选框架, 必要时以 A/B/C 上的完成率作为决胜标准。然后, 选定的候选框架在全部五个种子 A - E 上从头重新评估, 每个种子进行三次独立运行。

固定种子低样本量协议的理由。 BalatroBench 最初被设计为固定种子的模型评估设置, 而非作为框架演化的基准。我们采用固定种子, 是因为它们使候选比较在外层搜索期间可诊断: 如果每个候选都在不同的游戏实例上评估, 分数变化会将框架编辑与种子级变异混为一谈。同时, 在许多种子和多次展开上评估每个候选会使每个演化步骤变得缓慢且昂贵, 因为每个候选都需要完整的长期大语言模型智能体展开。因此, 我们的协议在搜索期间使用低样本开发预算, 并对选定的候选进行固定种子重新评估。这是一种工程权衡, 而非声称所得分数是平均 Balatro 技能的无方差估计。我们将固定种子结果与留出种子迁移、功能审计和行为分析结合起来解读。

条件特定输入。 BalatroLLM 是固定的 BalatroLLM 框架, 不进行外层演化, 仅在最终评估协议下评估。Meta-Harness 仅暴露先前框架代码的存档、开发种子分数、自生成展开和原始执行迹。OpenResearch 额外为提议者提供网络搜索访问, 并鼓励其搜索外部任务信息, 如规则、教程、策略指南或参考代码, 但不提供人类轨迹。DemoEvolve 提供九条有竞争力的人类轨迹, 包括每个开发种子 A/B/C 的三条轨迹。留出种子不提供人类轨迹。

D/E.

模型与对照。 所有 Balatro 任务执行智能体使用 GPT-5.4-low 作为冻结模型, 所有演化条件使用 Claude Opus 4.7 Max 作为编码提议者。在 Meta-Harness、OpenResearch 和 DemoEvolve 中, 初始框架、搜索预算、提议者模型、候选选择规则和最终评估协议均保持固定。

计算与应用程序编程接口成本。 我们使用 GPT-5.4-low 定价估算 Balatro 展开成本。最终固定种子评估使用五个种子, 每个种子三次展开, 即总共 15 次展开, 成本约为 32 美元, 即每次展开约 2.13 美元。一次典型的演化运行使用 10 次迭代, 每次迭代约有两个候选框架; 每个候选在三个开发种子上评估。这给出 $10 \times 2 \times 3 = 60$ 次开发展开, 对应约 128 美元。包括

最终固定种子评估所选框架，每次进化运行的总 rollout 成本约为 160 美元。该估算仅包括任务执行 rollout，不包括编码提议器的成本。

C.3 所选候选方案的功能审计

本附录审计了元框架自 rollout 基线在 A/B/C 开发种子选择规则下选出的最佳 Balatro 框架最终候选方案。所选候选方案为 face_exposure_shop_hook。它提出了一个针对依赖人头牌的构建的商店阶段钩子，旨在使“植物”（The Plant）这一使所有人头牌失效的 Boss 盲注之前警告智能体。源代码包含一个受保护的提示块：

```
{ % if hooks.face_exposure % } ... 人头牌暴露指标 ...
```

并且候选方案选择在 A/B/C 开发 rollout 中偏向此版本。

我们审计了渲染后的模型请求，以测试所提出的机制是否真正到达模型。在三个 A/B/C 验证 rollout 中，我们检查了 requests.jsonl 中的 517 个请求。没有出现任何独特的钩子字符串：

Expected hook string	Occurrences
Face-Card Exposure Metrics	0
Deck face cards	0
face-dependent jokers	0
The Plant boss disables	0

因此，尽管源代码差异非空，但所宣传的人头牌暴露干预并未出现在面向模型的上下文中。因此，候选方案在开发集上的增益不能归因于所提出的钩子。

这说明了 Balatro 中的噪声选择失败模式。即使使用相同的固定种子和几乎相同的提示，前沿大语言模型也可能产生不同的早期动作：一次商店购买、弃牌或打出的手牌都可能改变未来的状态分布和最终结果。在搜索过程中每个候选方案只有少数几次长 rollout 的情况下，当普通 rollout 方差使其得分看起来优于参考时，一个无效的编辑可能被选中。这是一种信用分配失败：优化器观察到更高的稀疏奖励并将其归因于候选方案差异，而执行上下文审计表明所声称的机制并未激活。

对于长时程的驾驭演化而言，源代码层面的差异因此是不充分的。选定的编辑应在渲染上下文或执行层面进行审计，以验证预期的提示块、钩子输出、记忆条目或工具模式在其应当生效的决策状态处确实处于激活状态。